



Phase 2.5 — Practical Assignments

Week 1 — Filesystem & Links

Assignment 1: Filesystem Forensics

Task:

1. Create a directory structure like

```
|--project/  
    |--src/  
    |--build/  
    |--log/
```

2. Create files inside `src/`

3. Create:

- One hard link
- One symbolic link

4. Delete the original file.

5. Observe behavior using:

- `ls -li`
- `stat`

Deliverable:

- Explain inode behavior
- Explain why one link survives and one breaks

Assignment 2: Permission Debugging Drill

You are given:

Permission denied

Simulate 3 different causes:

- Missing execute on directory
- Wrong ownership
- Incorrect file mode

Then fix them without using `sudo` unless necessary.

Assignment 3: Safe Backup Workflow

Create a script-less manual workflow that:

- Archives a directory
 - Compresses it
 - Stores it with timestamp
 - Verifies size before and after
-

Assignment 4: Shell Personalization & Alias Behavior

Task:

1. Create 5 useful aliases:
 - One for `ls`

- One for navigation
- One for search
- One for git (if installed)
- One custom productivity shortcut

2. Make them:

- Work in current session
- Persist across terminal restarts

3. Verify:

- `type <alias_name>`
- `alias`
- Restart terminal and test again
- Create an alias that breaks something intentionally.
Example:

```
alias rm='rm -i'
```

- Then override it temporarily using:

```
\rm
```

Deliverable:

- Explain why `\command` bypasses alias
 - Explain why aliases don't work inside non-interactive scripts
 - Show difference between alias and function
-

Week 2 — Processes & Signals

Assignment 1: Process Investigation

1. Start multiple background processes:

- `sleep`
- `yes > /dev/null`

2. Use:

- `ps`
- `top`
- `pstree`
- `lsof`

Explain:

- Process hierarchy
- CPU consumption
- Open file descriptors

Assignment 2: Signal Lab

1. Start a long-running process.

2. Send:

- `SIGINT`

- SIGTSTP
- SIGCONT
- SIGKILL

Document behavioral differences.

Assignment 3: PATH Hijack Experiment

1. Create a fake command in a custom directory.
2. Modify `PATH`.
3. Override system command behavior.

Explain why this is dangerous.

Week 3 — Streams & Redirection

Assignment 1: File Descriptor Mastery

Create commands that:

- Redirect stdout only
- Redirect stderr only
- Merge both
- Separate both

Then intentionally generate errors and observe output.

Assignment 2: Pipeline Engineering

Given a messy log file:

Extract:

- Unique error messages
- Count occurrences
- Sort by frequency

Must use:

- `grep`
- `sort`
- `uniq`
- `awk`

Assignment 3: Heredoc Script Input

Write a small script that:

- Accepts heredoc input
 - Processes lines
 - Outputs structured results
-

Week 4 — Text Processing & Networking

Assignment 1: Log Analyzer

Given:

- `/var/log/syslog` or any large log

Extract:

- Errors from last hour
- Count per service
- Top 5 frequent issues

Must combine tools.

Assignment 2: Network Debugging Drill

Using:

- `curl`
- `netstat / ss`
- `ping`

Diagnose:

- Whether a port is open
 - Whether a service is reachable
 - Whether DNS resolves
-

Assignment 3: Data Extraction Challenge

Given a CSV-like file:

- Extract column 2
- Filter rows
- Compute aggregate

Must use:

- `awk`
 - `cut`
 - `grep`
-

Week 5 — Compilation & Automation

Assignment 1: GCC Deep Dive

1. Compile a C program with:
 - Debug symbols
 - Warnings
2. Generate object file separately.
3. Link manually.
4. Use `ldd` to inspect dependencies.

Explain:

- What is linked dynamically
 - What is resolved at runtime
-

Assignment 2: Build Helper Script

Write a script that:

- Cleans build directory
- Compiles project
- Moves binary to `bin/`
- Prints success/failure based on exit code

Must use:

- `set -e`
 - `set -o pipefail`
 - Functions
-

Assignment 3: Automated Backup Script

Script must:

- Accept directory argument
- Validate input
- Create timestamped archive
- Handle errors

- Log actions